iPRA with equality at witness discharge

Three inference patterns and one explicit reflexivity axiom

Research prepared for Vladimir Reshetnikov

November 11, 2026

Abstract

We replace separate equality transport and existential witness elimination by one uniform rule: within a fresh witness scope, prove $b = t$ and a formula F, then export the normalized formula $F[t/b]$ if it no longer contains b. Ordinary elimination is the case $t = b$. Conversely, a finite naming construction recovers every ordinary equality transport. This is an exact equivalence of open consequence relations, not just an agreement on equations or closed sentences.

Combining the rule with the existing optional-premise clause representation gives three instruction families and one explicitly retained reflexivity axiom clause. The delivered ordinary realization is bi-interpretable and equiconsistent with full first-order intuitionistic primitive recursive arithmetic. Both numerical interpretation domains are whole; interpreted equality is actual equality; both numerical composite maps have graph $x = y$. The complete checker has 732 significant tokens, 813 AST nodes and 2,273 normalized UTF-8 bytes, compared with 769, 841 and 2,362 for a matched separate-rule control. Maximum-compression gzip sizes are 903 and 917 bytes. The smaller checker does not establish absolute minimality and does not beat the repository's separate 516-byte compact codec. A fuel-bounded verifier supplies a primitive-recursive certificate interface for the same theorems.

Contents

1 Result, comparison target, and exact scope **3**
1.1 The complete delivered presentation at a glance . . . 3
1.2 What the result does and does not establish . . . 4

2 Equality at the discharge of a numerical witness **4**
2.1 Syntax, normal forms, and the four-pattern presentation . . . 4
2.2 Direct soundness of the fused inference . . . 5
2.3 What can be recovered before any equality reasoning . . . 5
2.4 Equality recovered by a two-row witness package . . . 6
2.5 Repair of atomic case introduction at the zero witness . . . 7
2.6 Exact sequent equivalence and the interpretation boundary . . . 8
2.7 Three instruction patterns with an explicit fixed axiom clause . . . 9

3 Complete checker comparison and implementation evidence **10**
3.1 Measurements and matched controls . . . 10
3.2 The public certificate contract . . . 11
3.3 The implementation invariant . . . 11
3.4 A primitive-recursive certificate interface . . . 11
3.5 Executed verification and its limits . . . 12

A Computation normalization and the raw comparison **13**
A.1 Normalization preserves provable formulas internally . . . 14
A.2 Uniform evaluation is not an object-language operation . . . 15

Attribution and role of the following foundations **16**

B Complementary introductions with one local assumption **16**
B.1 Complete native syntax and rule inventory . . . 16
B.2 Exact equivalence and the unused-witness construction . . . 17
B.3 Induction and numerical cases separate cleanly . . . 18
B.4 The entire ordinary logical interface . . . 19
B.5 Whole-domain realizations and both formula composites . . . 20
B.6 What the theorem does and does not claim . . . 21

C Arithmetic strength and both ordinary interpretation composites **21**
C.1 The arithmetic dependency order . . . 22

D Internal scalar predecessor bootstrap **23**
D.1 Elementary terms and two finite cycles . . . 24
D.2 A ramp with a global scaling identity . . . 24
D.3 A comparator from scalar iteration . . . 25
D.4 Saturated iteration gives predecessor . . . 26
D.5 Dependency boundary . . . 27

E From literal-equation induction to full intuitionistic PRA **27**
E.1 The exact hypotheses . . . 28
E.2 Constructor separation and two elementary tests . . . 28
E.3 Four elementary uses of equation induction . . . 29
E.4 An equation invariant that reflects actual equality . . . 30
E.5 Compilation of all quantifier-free formulas . . . 31
E.6 Recovery of induction and the full theory . . . 32

F Pairing, primitive recursion, and internal numerical composites **33**
F.1 Pairing from a bounded square-root search . . . 33
F.2 Compiling each primitive recursion . . . 34
F.3 Why both numerical round trips are internal . . . 34

G Complete normalized acceptance checker **35**

H Reproduction and file map **36**

1 Result, comparison target, and exact scope

The substantive simplification is the removal of a separate equality elimination instruction. Equality is used at the point where a numerical witness is discharged. This is one uniform inference with three premises, not a tagged union of an equality case and a logical case.

The comparison snapshot is the ProveIt repository at `0ebd0c6b358f`, including its six incoming archives. We also inspected the subsequent commit `443cff495634`, which formalizes the already existing zero-normal single-Step interface and does not add the present discharge rule. The scoped novelty search covered the filed research source texts and those incoming archives. It supports the claim that this fusion was absent from the inspected proposal. It is not a claim of priority over every axiomatization in the literature.

The scalar iterator, the paired binder D, the logical macros, the arithmetic-to-iPRA bridge, full computation normalization, the naming packet, and the optional-premise clause representation are inherited. Their sources are identified in the bibliography and in `SOURCE_PROVENANCE.json`. The new result is the equality-aware discharge rule and its exact elimination of the separate transport rule.

1.1 The complete delivered presentation at a glance

There is one numerical sort. Terms are full normal forms of $x, 0, St, I(n, a; u.h)$; the two defining reductions of I are stated in Section 2. Formulas are equations and $D_{x,y}B$. A reusable clause $[\vec{A}; y.B]$ has either no antecedent or one antecedent, and y is absent from its antecedent. An ordinary assumption F is the unconditional constant clause $[\,; v.F]$. Every proof starts with

$$\boxed{\mathcal{R} = [\,; y.y = y].}$$

This is the only explicit initial axiom clause of the native presentation; it is an explicit reflexivity axiom, not a theorem obtained from nothing. The native contradiction is $0 = S0$.

There are exactly three instruction families:

1. **Use a clause.** For a retained $[\vec{A}; y.B]$, prove its single antecedent when present, and conclude $\mathrm{nf}(B(t))$ at any numerical term t. An unconditional clause has no proof premise.

2. **Introduce a paired formula.** From $\Gamma, B(t, \eta) \vdash B(t, Sz)$ infer $\Gamma \vdash D_{x,y}B(x, y)$, where z is fresh and $\eta = z$ precisely when B has equation root and the normal witness t is literally 0; otherwise $\eta = 0$.

3. **Discharge a witness with equality.** From $D_{x,y}B$, open its clause $[B(b, 0); y.B(b, y)]$ at fresh b. In that same extended context prove $b = t$ and F. Export $\mathrm{nf}(F[t/b])$ only if the exported formula is free of b.

All displayed substitution instances are normalized. The substitution in the third rule is simultaneous, and its target may contain b. In particular, $t = b$ is permitted. Freshness includes all retained clauses and all passive parameters of the major formula. There are no additional primitive logical, successor, equality, case, induction, weakening, or cut rules. Full numerical normalization is a trusted syntactic operation, included in every complete checker measurement.

For the proofs we first expose reflexivity as an explicit rule, obtaining an auxiliary four-pattern calculus. Section 2.7 then proves the exact translation to the three-pattern implementation above. This separation keeps the source of reflexivity visible.

1.2 What the result does and does not establish

The new presentation proves the same complete first-order intuitionistic theory as the reference presentation. It does not add induction for arbitrary formulas: only literal-equation progress is primitive. Both interpretation directions and both composites are addressed, including the logical connectives and quantifiers; equiconsistency is not inferred merely from standard-natural-number behavior.

No absolute smallest presentation is claimed. A metric based on a program's size depends on the language, input contract, representation, and normalization convention. The paper establishes a structural deletion and a strict matched checker improvement. The repository also contains much shorter compact representations of its earlier calculus, so this is not a new global source-size record. The mathematics is a written universal proof with executable examples and independent replay, not a new Lean or Rocq formalization.

2 Equality at the discharge of a numerical witness

The change is a uniform strengthening of paired elimination. Equality substitution is performed only while discharging the fresh numerical witness of that elimination. An arbitrary equality inference elsewhere in a proof is recovered by first giving its source a fresh name. The naming construction is inherited from the variable-substitution proposal [2]; its final substitution and witness discharge become one primitive inference here.

2.1 Syntax, normal forms, and the four-pattern presentation

Write N for full computation normalization of finite scalar-iterator terms

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h),$$

under the two context-closed reductions

$$I(0, a; u.h) \longrightarrow a, \qquad I(Sn, a; u.h) \longrightarrow h[I(n, a; u.h)/u].$$

Only u in h is bound. All substitutions avoid capture. Numerical terms of the calculus are the normal forms; every substituted instance is normalized. In particular, $I(n, a; u.h)$ in a displayed formula denotes the normal form of that constructor expression. Formulas and reusable clauses are

$$F ::= (s = t) \mid D_{x,y}F, \qquad [A; y.B(y)], \qquad y \notin \mathrm{FV}(A).$$

Normalization acts on numerical subterms and does not delete formula binders. A context is a finite list of clause occurrences. Put $H = (0 = 0)$, and abbreviate an ordinary formula assumption F by the constant clause $[H; v.F]$, with v unused. Context membership is by occurrence, so equal-valued clauses may still have distinct positions.

The presentation $\mathcal{F}_4$ has the following four inference patterns.

$$\frac{}{\Gamma \vdash t = t}. \tag{R}$$

$$\frac{\Gamma \vdash A}{\Gamma \vdash N(B(t))}, \qquad [A; y.B(y)] \in \Gamma. \tag{U}$$

$$\frac{\Gamma, B(t,\eta) \vdash B(t,Sz)}{\Gamma \vdash D_{x,y}B(x,y)}, \qquad \eta = \begin{cases} z, & B \text{ has equation root and } t \equiv 0, \\ 0, & \text{otherwise.} \end{cases} \tag{I}$$

Here z is fresh for Γ, t, and all passive matrix parameters. All instances in this display are canonical; $t \equiv 0$ is literal equality of canonical syntax. Finally,

$$\frac{\Gamma \vdash D_{x,y}B(x,y) \quad \Gamma, \mathcal{B}_b \vdash b = t \quad \Gamma, \mathcal{B}_b \vdash F}{\Gamma \vdash N(F[t/b])}, \qquad \mathcal{B}_b = [B(b,0); y.B(b,y)]. \tag{$E_=$}$$

The parameter b is fresh for Γ and the passive parameters of B. The two branch proofs have the *same* extended context. The term t and formula F are well scoped in that branch, and the exported formula $N(F[t/b])$ must be free of b. Thus the exported formula is well scoped in the outer numerical context. The substitution is simultaneous: an inserted copy of t is not traversed again.

In particular, t is allowed to contain b. The special choice $t = b$ is essential below. The condition concerns the exported normal form, not a syntactic prohibition of b in t. These are all the primitive inferences. There is no additional equality inference, computation axiom, logical axiom, successor axiom, or induction rule. The two computation equations are built into canonical syntax and substitution; their normalizer is part of the trusted presentation and the measured checker.

2.2 Direct soundness of the fused inference

The ordinary readings are

$$(D_{x,y}B)^\circ = \exists x\big(B^\circ(x,0) \to \forall y\, B^\circ(x,y)\big), \qquad [A; y.B]^\circ = A^\circ \to \forall y\, B^\circ(y).$$

Equality has its ordinary meaning. For soundness in the inherited ordinary scalar theory, rule I uses numerical coverage in its case mode and literal- equation induction in its progress mode. R and U have their indicated usual readings. The only changed inference is $E_=$.

Lemma 2.1 (Soundness of witness discharge with substitution)**.** *Rule* $(E_=)$ *is sound in intuitionistic first-order equality logic over the inherited scalar arithmetic, for arbitrary* F*, including compound formulas, and also when* t *contains* b*.*

Proof. Eliminate the existential in the ordinary reading of the major premise at fresh b. Its local assumption is precisely the reading of $\mathcal{B}_b$. The two supplied branch derivations give $b = t$ and F. Ordinary equality substitution gives $F[t/b]$, and the iterator computation equations identify this formula with its normal form. By the rule's side condition that normal form has no free b, so ordinary existential elimination discharges b and its clause.

This equality step does not require $b \notin \mathrm{FV}(t)$. For example, equality $b = t(b)$ supports simultaneous substitution of $t(b)$ for b in any fixed formula. The condition needed for existential elimination is the absence of b from the *result*, which the rule requires explicitly. □

2.3 What can be recovered before any equality reasoning

Lemma 2.2 (Ordinary paired elimination)**.** *The ordinary rule*

$$\frac{\Gamma \vdash D_{x,y}B(x,y) \qquad \Gamma, \mathcal{B}_b \vdash A}{\Gamma \vdash A}, \qquad b \notin \mathrm{FV}(\Gamma, A),$$

is derivable in $\mathcal{F}_4$*.*

Proof. Take $t = b$ in $E_=$. Its equality branch is reflexivity $b = b$, and its other branch is the given proof of A. Since A has no free b, $N(A[b/b]) = A$ literally. □

Weakening, injective fresh renaming, and ordinary-assumption cut are admissible by direct induction on derivations. Freshen the eigenparameters of I and $E_=$ before inserting proofs. The substitution source in $E_=$ is the parameter bound by that very inference; inserting additional parameters outside its scope does not change that source. A cut of an ordinary assumption replaces uses of its constant clause by the supplied proof. These arguments require no arbitrary numerical substitution theorem for derivations and no equality congruence.

For an unused w, abbreviate $q_y A(y) = D_{w,y} A(y)$.

Lemma 2.3 (Closure and numerical cases without equality)**.** *Using only R, U, I, and derived ordinary paired elimination, one obtains*

$$\frac{\Gamma, A(0) \vdash A(Sz)}{\Gamma \vdash q_y A(y)}, \qquad \frac{\Gamma \vdash q_y A(y) \qquad \Gamma \vdash A(0)}{\Gamma \vdash A(t)}.$$

Consequently, from $\Gamma \vdash A(0)$ and $\Gamma \vdash A(Sz)$, with z fresh for the retained context and passive parameters, one obtains $\Gamma \vdash A(t)$ for every well-scoped term t.

Proof. For the first rule introduce the unused witness coordinate at $1 = S0$. Rule I is then in case mode at every matrix root. For the second rule use ordinary paired elimination: since w is unused, its opened clause is $[A(0); y.A(y)]$. Rule U with the supplied zero proof and target t gives the result. To get numerical cases, weaken the successor proof by the unused assumption $A(0)$ and compose the two constructions. □

2.4 Equality recovered by a two-row witness package

The inherited normalizing selector and parity terms are

$$C(v, a, c) = N(I(v, a; u.c)), \quad u \notin \mathrm{FV}(c), \qquad P(y) = N(I(y, 0; u.C(u, 1, 0))).$$

Their endpoint computations are literal canonical-syntax identities:

$$C(0, a, c) \equiv a, \quad C(Sv, a, c) \equiv c, \qquad P(1) \equiv 1, \quad P(2) \equiv 0.$$

No induction or range theorem for P is needed.

Theorem 2.4 (Uncrossed transport from one fused discharge)**.** *For every fixed numerical template $h(w)$, the inference*

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash N(h(s)) = r}{\Gamma \vdash N(h(t)) = r}$$

is derivable in $\mathcal{F}_4$. Apart from ordinary paired eliminations recovered by $t = b$, it uses one equality-bearing fused discharge.

Proof. Choose x, y fresh for all passive data and define the equation matrix

$$B(x, y) = \big(C(y, 0, C(P(y), x, N(h(x)))) = C(y, 0, C(P(y), t, r))\big).$$

Introduce $D_{x,y} B(x, y)$ at witness s. Its successor instance is

$$C(P(Sz), s, N(h(s))) = C(P(Sz), t, r).$$

Apply Lemma 2.3 to the formula

$$A(v) = \big(C(v, s, N(h(s))) = C(v, t, r)\big)$$

at target $P(Sz)$. Its zero instance is the supplied $s = t$, and every successor instance is the supplied $N(h(s)) = r$. Choose the case eigenparameter fresh for the current local context, independently of z. This derivation ignores the temporary assumption of I. It therefore works whether s is literal zero and I uses progress, or I uses cases.

Now apply $E_=$ to this package with fresh parameter b. The opened clause is $[H; y.B(b, y)]$, since $B(b, 0) \equiv H$. By U at 1 and 2, using reflexivity for H, obtain the literal normalized instances

$$B(b, 1) \equiv (N(h(b)) = r), \qquad B(b, 2) \equiv (b = t).$$

Use the latter as the equality branch of $E_=$, and use $F = (N(h(b)) = r)$ as its formula branch. Normalization–substitution coherence identifies the exported formula as

$$N\big((N(h(b)) = r)[t/b]\big) \equiv (N(h(t)) = r).$$

The fresh b does not occur in t, r, h, or the outer context, so the exported formula is outer scoped. This identity is a metasyntactic coherence identity, not an invocation of an unproved equality rule. □

Corollary 2.5 (Symmetry, crossed transport, and ordinary equality)**.** *The presentation derives symmetry, transitivity, term congruence, crossed transport $s = t, h(s) = r \Rightarrow r = h(t)$, and transport in every formula template.*

Proof. In Theorem 2.4, take $h(w) = w$ and $r = s$; the premises $s = t$ and reflexivity $s = s$ give $t = s$. Thus symmetry is obtained without assuming it in the naming package. Uncrossed transport followed by symmetry gives crossed transport. The standard two applications of crossed transport give replacement in both sides of an equation; symmetry and root joining give transitivity, and a reflexive minor gives congruence.

For transport through a compound template $D_{x,y}B(v, x, y)$, argue by external induction on formula depth. From $s = t$ and $D_{x,y}B(s, x, y)$, use ordinary paired elimination at a fresh c. Introduce $D_{x,y}B(t, x, y)$ at witness c, which is not literal zero. Under its local assumption $B(t, c, 0)$, the induction hypothesis and symmetry give $B(s, c, 0)$. Specialize the old opened clause at Sz, obtaining $B(s, c, Sz)$, and transport forward by the induction hypothesis to $B(t, c, Sz)$. Close I and ordinary elimination. Every recursive transport uses a strictly smaller formula. This argument does not require induction on quantified formulas. □

2.5 Repair of atomic case introduction at the zero witness

Full computation normalization removes closed nonliteral aliases for zero. The inherited compound-envelope construction repairs the missing case without such an alias; it remains available in the new presentation before using Theorem 2.4.

For an equation $E = (s = t)$, put

$$G_vE = (C(v, 0, s) = C(v, 0, t)), \qquad L(E) = q_vG_vE.$$

The gate has literal endpoints $G_0E \equiv H$ and $G_{Sw}E \equiv E$. Lemma 2.3 gives $E \vdash L(E)$ by closure introduction, and $L(E) \vdash E$ by closure specialization at 1 and reflexivity of H. These are uniform derivations under any retained context.

Suppose $E(x, y)$ is an equation and a supplied derivation proves $\Gamma, E(0, 0) \vdash E(0, Sz)$. Set $A(x, y) = L(E(x, y))$. Under $A(0, 0)$, unwrap the envelope, insert the supplied derivation by ordinary-assumption cut, and wrap the resulting equation. This proves $A(0, Sz)$. Rule I at witness zero now uses cases because A is compound, so it gives $D_{x,y}A(x, y)$.

Open this formula by ordinary elimination at a fresh c. To introduce $D_{x,y}E(x, y)$ at witness c, assume $E(c, 0)$, wrap it as $A(c, 0)$, use the opened clause at Sz, and unwrap $A(c, Sz)$ to $E(c, Sz)$. The witness c is not literal zero, so I again uses cases. Closing both inferences yields $D_{x,y}E(x, y)$. Hence unrestricted case introduction is a derived rule. This is a separate completeness repair, not an extra primitive induction principle.

2.6 Exact sequent equivalence and the interpretation boundary

Let $\mathcal{T}_5$ be the same canonical Eq/D clause calculus with R, U, I, ordinary paired elimination, and crossed term transport as its five patterns. This is the full-normalization clausal control.

Theorem 2.6 (Exact identity-language equivalence)**.** *For every finite canonical clause context* Γ *and canonical formula* F*,*

$$\Gamma \vdash_{\mathcal{F}_4} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{T}_5} F.$$

Both directions are effective transformations of finite proof trees and leave the entire end sequent literally unchanged.

Proof. Translate a $\mathcal{T}_5$ proof recursively. Replace ordinary elimination by Lemma 2.2, and replace each crossed transport by Theorem 2.4 followed by the derived symmetry construction. R, U, and I are unchanged. The preliminary numerical-case constructions used by the naming package were derived without either equality transport or a translated transport node, so the recursive translation is well founded.

Conversely, the control derives whole-formula equality transport by the same formula-depth argument as Corollary 2.5. In a translated $E_=$ branch, apply that transport to $b = t$ and F, obtaining $N(F[t/b])$. Its stipulated b-freedom permits ordinary paired elimination. All other patterns are shared. Freshen the finitely many local parameters before each replacement. □

Composing Theorem 2.6 with the attributed canonical/raw comparison and the scalar/iPRA arithmetic bridge gives full first-order intuitionistic PRA, not merely its equations or closed sentences. Numerical interpretations use the whole numerical domains and actual equality. The new comparison is the identity on terms and formulas, so it preserves the inherited definable numerical composite isomorphisms with graph $x = y$ and both inherited formula round trips. Equiconsistency follows in particular because the common designated contradiction $0 = S0$ is unchanged.

Normalization is metasyntactic. The comparison uses the established termination, confluence, and substitution coherence of the iterator reductions. Canonical syntax recognition itself is a structural check. Normalizing a supplied substituted instance is a separate trusted computation and must be included in the complete checker size. There is no numerical variable ranging over function descriptions and no object-language uniform evaluator or totality axiom. Uniform normalization over arbitrary primitive-recursive descriptions has no primitive-recursive bound in general; the standard diagonal obstruction to a universal primitive-recursive evaluator applies. The proof translations above are claimed to be effective, not automatically primitive recursive. Explicit computation traces would give a different certificate interface and would need to be counted separately.

Substitution of derivations. A primitive-tag-preserving numerical-substitution theorem is not assumed: normalization can turn a witnessing term into literal zero and thereby change I's selected mode. The envelope repair or transfer through the substitution-stable canonical reference calculus supplies the corresponding admissibility result. The substitution inside $E_=$ is an operation on its proved formula, justified by equality, not an attempt to substitute blindly into the branch derivation.

2.7 Three instruction patterns with an explicit fixed axiom clause

Using the optional-premise interface of [3], there is also a uniform-clause presentation $\mathcal{F}_3$. Its contexts contain clauses $[\vec{A}; y.B(y)]$ with zero or one antecedent. Replace U by

$$\frac{\{\,\Gamma \vdash A : A \text{ is the antecedent occurrence in } \vec{A}\,\}}{\Gamma \vdash B(t)}. \tag{$U_{0,1}$}$$

For an empty antecedent list this inference has no premise. Every proof starts with the one explicit fixed axiom clause

$$\mathcal{R} = [\,; y.y = y]. \tag{Ax}$$

Ordinary local assumptions F are now unconditional constant clauses $[\,; v.F]$. The I pattern is otherwise unchanged. The $E_=$ pattern opens the singleton-premise clause $[[B(b,0)]; y.B(b,y)]$ and has exactly the same two branch conclusions, substitution, and scope condition. There is no R instruction: $t = t$ is a use of $\mathcal{R}$ at t.

Theorem 2.7 (Uniform clauses preserve all consequences)**.** *The three instruction patterns $U_{0,1}$, I, $E_=$, together with the fixed axiom clause $\mathcal{R}$, give the same theory as $\mathcal{F}_4$. The comparison covers arbitrary retained clauses with zero or one antecedent, not only clauses reachable in a closed proof.*

Proof. For a context Δ of optional-premise clauses, define $L(\Delta)$ occurrence by occurrence by

$$[\,; y.B] \longmapsto [H; y.B], \qquad [[A]; y.B] \longmapsto [A; y.B].$$

Exclude the distinguished initial axiom occurrence $\mathcal{R}$ from this translation; its uses are translated to R. Then for every F,

$$\mathcal{R}, \Delta \vdash_{\mathcal{F}_3} F \quad \Longleftrightarrow \quad L(\Delta) \vdash_{\mathcal{F}_4} F.$$

Forwards, an unconditional U use becomes an ordinary U use supplied with R at zero. A singleton U use is unchanged. The clauses opened by $E_=$ correspond exactly, as do I's local assumptions after this translation. An axiom-clause use becomes R at its selected term. Translate the two branch proofs of each fused elimination recursively.

Backwards, R is an empty-premise use of $\mathcal{R}$. For a U use selecting a clause that came from an unconditional occurrence, omit the supplied proof of H; singleton occurrences keep their premise proof. Local ordinary assumptions introduced by I correspond to unconditional clauses in $\mathcal{F}_3$, so their U uses are handled in the same way. The fused elimination and introduction have unchanged targets and side conditions. Retain the origin of each clause occurrence during this translation, including when different origins have the same clause value.

The same argument applies to matched controls that have separate equality transport and ordinary paired elimination. Thus the fusion theorem holds with the optional-premise context syntax as well. All end formulas remain identical, and the ordinary meanings of corresponding contexts agree. □

The number three counts executable inference patterns after reflexivity has been moved to one explicitly listed initial axiom clause. It does not assert that reflexivity was proved without an axiom. The mathematical reduction specific to this report is the localization of all equality elimination to the discharge of a numerical witness. The optional-premise clause interface and the initial reflexivity clause are inherited choices of presentation. Neither rule counting nor a smaller matched checker is a proof of an absolutely simplest axiomatization over all languages, codings, or implementations.

3 Complete checker comparison and implementation evidence

3.1 Measurements and matched controls

Every row below measures a complete standalone module, including syntax validation, substitution, computation normalization, context handling, input decoding, and the acceptance/rejection entry point. The convention is the repository's CPython 3.12.14 convention: remove the module docstring, apply `ast.unparse`, and append one LF. Significant tokens exclude comments, indentation and newline tokens; AST nodes are counted by `ast.walk`. Gzip uses level 9 and timestamp zero, and is informational rather than an optimization target.

Complete checker	Bytes	Tokens	AST	Gzip-9
Witness discharge	2,273	732	813	903
Matched separate rules	2,362	769	841	917
Witness discharge, fuel	2,486	777	868	967
Matched control, fuel	2,575	814	896	980
Incoming single evaluation	2,283	741	826	818
Incoming variable substitution	2,840	905	976	1,058

The matched control uses the same term and formula encoding, full normalizer, optional-premise clauses, initial reflexivity clause, goal-directed checking, and public closed-goal contract. Its equality rule is unrestricted crossed transport with one term template, and it has ordinary paired elimination as a separate instruction. The fusion saves 89 bytes (3.77%), 37 tokens (4.81%), and 28 AST nodes (3.33%). Gzip decreases by 14 bytes.

The incoming single-evaluation checker is a useful stronger external comparison: the new module is smaller by 10 bytes, 9 tokens and 13 AST nodes, while gzip grows by 85 bytes. That comparison changes normalization as well as the inference interface; it does not isolate the new theorem. The matched control does isolate it. The incoming variable-substitution row has a more verbose tagged syntax and is included for provenance, not as the sole comparison target.

The published compact codec measures 516 bytes, 265 tokens, 306 AST nodes and 332 gzip bytes when run through the same source-measurement convention. The present package does not beat it. Its compact raw-object protocol is a different implementation family; a short proof-system inventory does not imply a smaller implementation in every representation. We therefore do not extrapolate the matched improvement into a global minimum claim.

The readable delivered checker contains comments and a substantial interface docstring: its source is 3,593 bytes and gzip-9 is 1,566 bytes. Normalized source and its gzip file are delivered alongside it. The distinction is explicit; comments and formatting have not been counted as mathematical simplification. All rows and source hashes can be reproduced by `python3 measure.py`.

3.2 The public certificate contract

The ordinary entry point reads one JSON object with exactly two keys, `goal` and `proof`. The goal must be a closed canonical formula. Integers encode numerical syntax: 0 is zero, negative integers are free parameter levels, and positive integers are one-based bound indices. A successor is `[0,t]`, an iterator is `[n,a,h]`, an equation is `[s,t]`, and a paired formula is `[B]`. An iterator binds one variable only in its final child; a paired formula binds two variables. In a D matrix, index 1 is the universal coordinate and index 2 is the existential coordinate.

The three proof packets are:

```
["u", index, term, premise]
["i", witness, branch]
["e", matrix, local_formula, target, major, equality, branch]
```

The selected goal is supplied by the current recursive call. Context clauses are stored in order of insertion. The initial reflexivity clause has index zero. For unconditional Use, the premise payload is intentionally ignored; `null` is recommended. This is a nullary inference, not unchecked evidence for an antecedent. For a conditional clause the antecedent proof is mandatory. The entry point prints `ACCEPT` only after verification; any exception prints `REJECT` and exits unsuccessfully.

No untrusted proof producer, translator, test, or reference module is imported by the acceptance checker. Its only imports are `json` and `sys`. The mathematical theorem covers arbitrary canonical open clause sequents; the public command-line interface deliberately requests only closed theorems.

3.3 The implementation invariant

At every successful call of `prove`, the goal has been fully validated at the current numerical scope, and each retained clause is well scoped and canonical. The shared traversal validates all children before a computation can erase them. Thus malformed data cannot be hidden in an unused iterator body or seed. Normalization of an internal successor iterate uses genuine capture-avoiding hereditary substitution, including the lifting needed when an inserted term contains indices bound outside an inner iterator.

In a witness-discharge call the major is checked first. The new clause is constructed by opening its already checked matrix. The equality branch is checked against exactly $b = t$; therefore its target is a well-scoped canonical term in the extended scope. The other branch is checked there as well. Only then does the checker substitute for the negative free level b. The inserted target has no loose positive indices, so moving it beneath an object-language binder does not capture it. The traversal never revisits an inserted copy. Finally its result must equal the already checked outer goal; this enforces the required absence of the eigenparameter even under nested paired binders or after normalization erases an occurrence.

Generic calls of `walk` omit a redundant root sort check because their inputs are checked syntax, syntax preserved by substitution, or a predecessor iterate constructed from checked components. Direct certificate syntax always enters with a Boolean sort. This successful-derivation invariant is part of the code argument; `walk` is not advertised as a public parser for arbitrary untyped trees.

3.4 A primitive-recursive certificate interface

Uniform normalization on all primitive-recursive term descriptions is not primitive recursive; Appendix A explains the distinction. The smaller checker is therefore a total-recursive mathematical

verifier, with potentially very large computation. It is not an internal uniform evaluation operation of iPRA.

For applications that require a primitive-recursive certificate predicate, `checker_fuel.py` adds a nonnegative integer `fuel` field. Every entry into the recursive syntax/normalization traversal or proof checker consumes one unit. Insufficient fuel rejects. All remaining loops operate on finite explicit lists. A bound on recursive calls bounds expansion by a primitive-recursive function of the input size and fuel, so the bounded acceptance relation is primitive recursive. This claim concerns the finite algorithm, not a uniform totality assertion for the unbounded normalizer.

Every accepted unbounded certificate uses finitely many such calls and therefore has an accepting fuel annotation. Conversely, fuel can only reject an otherwise valid proof. Thus the bounded verifier accepts exactly the same theorems when fuel is existentially supplied as part of a certificate. Its matched control contains the identical budget mechanism, preserving the 89-byte, 37-token and 28-node advantage. The budget is an implementation parameter, not a new arithmetic or logical axiom.

As usual, these algorithmic completeness and primitive-recursiveness statements use an idealization with unbounded finite storage (or an explicit-stack simulation). The concrete Python implementation has finite stack and memory limits. It catches resource exceptions and rejects when those limits are exceeded, so adequate mathematical fuel alone does not guarantee success on fixed physical hardware.

3.5 Executed verification and its limits

The delivered evidence includes 13 substantive closed theorem certificates and two explicit tests of the unconditional-payload convention. Each is checked by the selected implementation, translated to a separately implemented forward calculus and replayed independently, and translated in both directions through the matched separate-rule control. Both theorem and exact-goal round trips pass. The fixtures include symmetry, transitivity using two equality assumptions, congruence with repeated distinguished occurrences, preservation of passive occurrences of the same source term, nested iterator binders, genuine use of an induction hypothesis, and a zero-witness case repair that consumes its base assumption. One additional open sequent checks a passive numerical parameter.

All 36 adversarial certificates are rejected. These test wrong equality orientation, wrong exported formulas, conditional-clause antecedents, eigenparameter escape under binders, ill-scoped terms, booleans disguised as indices, malformed syntax, and noncanonical terms. The ordinary and optimized Python command-line entry points pass 116 checks, including malformed JSON documents and headers. A further audit supplies 221 normal-form comparisons, 660 numerical evaluations and 440 binder-opening checks with no skips.

An independent named-variable normalizer and an independent natural-number environment evaluator give 2,505 exact normal-form comparisons and 15,027 numerical comparisons. There are also 1,000 bound-substitution comparisons, 500 free-name substitutions (including targets containing the source), 400 substitutions through nested D formulas, and 17 malformed-term rejections. Three numerical evaluations exceed that audit's explicit resource bound and are recorded as skipped; no exact normal-form comparison is skipped. The capture regression

$$I(1,1;a.I(1,a;u.I(a,0;w.u))) \quad \longmapsto \quad 1$$

passes. A naive substitution that fails to lift the inserted outer variable can incorrectly compute zero here, so this is a substantive binder check.

The fuel variants pass 28 accepted examples across the two interfaces, their exact-budget and one-unit-short checks, 12 malformed-budget rejections, and 112 public command-line checks. These are finite implementation tests, not machine verification of the universal metatheorems. The paper's general claims rest on the displayed proofs and the attributed arithmetic bridge.

A Computation normalization and the raw comparison

This appendix adapts the normalization argument in the repository's `full_normalization` package [5]. It is included because moving computation into syntax must not hide a metatheoretic assumption or uncounted algorithm. Let reduction be the contextual closure, including under iterator updates, of

$$I(0, a; u.h) \to a, \qquad I(Sn, a; u.h) \to h[I(n, a; u.h)/u].$$

Capture-free substitution preserves reduction. For a strongly normalizing finite term t, let $\nu(t)$ be the maximum length of a reduction from it. This maximum exists because the reduction tree is finitely branching and well founded. Write $|t|$ for its finite syntax size.

Lemma A.1 (Iterator closure)**.** *If n, a are strongly normalizing and $h[v/u]$ is strongly normalizing for every strongly normalizing term v, then $I(n, a; u.h)$ is strongly normalizing.*

Proof. The body h is strongly normalizing by taking $v = u$. Induct lexicographically on $(\nu(n), |n|, \nu(a) + \nu(h))$ and inspect one-step reducts. A reduction in n decreases the first coordinate; a reduction in a or h decreases the last. For $h \to h'$, substitution stability preserves the universal hypothesis on the body. The zero root reduct is a. At a successor root $n = Sp$, the counter p has the same reduction height and smaller size. Hence $I(p, a; u.h)$ is strongly normalizing by the induction hypothesis, and substituting it into h is strongly normalizing by the premise on h. Every reduct is therefore strongly normalizing. □

Theorem A.2 (Strong normalization, confluence, and coherence)**.** *Every finite raw term is strongly normalizing and has a unique normal form, up to bound-variable renaming. With de Bruijn indices uniqueness is literal. For every substitution σ,*

$$\mathrm{nf}(t[\sigma]) = \mathrm{nf}\big(\mathrm{nf}(t)[x \mapsto \mathrm{nf}(\sigma x)]\big).$$

The analogous identity holds for formulas.

Proof. Prove the stronger assertion that substituting strongly normalizing terms for the free variables of any fixed term yields a strongly normalizing term. Structural induction handles variables, zero and successor. At an iterator the induction hypotheses give the counter and seed premises of the preceding lemma, and the body induction hypothesis applies to every additional strongly normalizing assignment for its bound state variable. Taking the identity substitution gives strong normalization.

Reduction is locally confluent: disjoint contractions commute, the two root patterns do not overlap, and an inner contraction can be performed on each of its finitely many residual copies after a root contraction. Erased residuals need no step. Substitution stability gives a common reduct in the copying cases. Strong normalization and local confluence then give confluence by induction on reduction height.

Reduce t to $\mathrm{nf}(t)$ after substitution, and reduce each substituted argument to its normal form. The two resulting terms are convertible; unique normal forms give the coherence identity. Applying

it twice gives composition of normalized substitutions. Formula constructors do not reduce, so the same conclusion follows structurally for formulas. □

The delivered traversal first processes children, and, on a successor counter, normalizes the predecessor iterate before inserting it into the update. Every contraction is one of the displayed contextual reductions; the remaining operations are finite traversal and capture-avoiding index renaming. The theorem proves termination and identifies its result with nf. Normal forms have no iterator whose counter is headed by 0 or S.

A.1 Normalization preserves provable formulas internally

Let $\mathcal{C}$ be the attributed raw Choice-DI calculus described in Appendix B. It already derives equation induction, ordinary equality, and numerical cases. Transport in a numerical parameter does not by itself authorize capture of that parameter by an iterator binder. The following lemma supplies the needed update congruence.

Lemma A.3 (Update extensionality)**.** *If u is fresh for Γ and $\Gamma \vdash h(u) = k(u)$, then $\Gamma \vdash I(n, a; u.h) = I(n, a; u.k)$ for every counter and seed.*

Proof. Induct on the single equation comparing the two iterates at a fresh counter z. At zero both equal a. In the successor step, computation gives $h(I(z, a; h))$ and $k(I(z, a; k))$. Instantiate the supplied pointwise equality at the first iterate, and then use the induction equation in the fixed template k to identify these expressions. This is literal-equation induction with passive parameters fixed. □

Each contextual computation step is now a provable equality in $\mathcal{C}$: the root cases are its defining equations, ordinary transport handles successors, counters and seeds, and update extensionality handles descent under an update binder. Consequently every fixed finite term satisfies $t = \mathrm{nf}(t)$ internally. Equality transport and binder congruence yield $A \dashv\vdash \mathrm{nf}(A)$ for every fixed formula.

Theorem A.4 (Raw and canonical comparisons)**.** *Normalizing all terms in a raw sequent gives exactly the corresponding consequences of the canonical four-pattern calculus, and hence of the delivered three-pattern calculus. The canonical-to-raw map regards a normal term as raw syntax. The composite on canonical syntax is literal identity; the other composite is internally provable normalization equivalence.*

Proof. Forwards, first use an auxiliary canonical calculus with separate case introduction and equation-progress introduction. Normalize a raw proof node by node. Computation equations become reflexivity, and substitution coherence gives the remaining normalized instances. Translate each auxiliary case introduction by the envelope repair in Section 2.5. To recover progress at an arbitrary witness t, first introduce $D_{w,y}E(t, y)$ at witness zero, with unused w. Eliminate this formula to obtain its conditional closure and use it under the base $E(t, 0)$ to supply the successor premise for case introduction of $D_{x,y}E(x, y)$ at t. Thus both raw choices are recovered. The crossed transport is recovered by Theorem 2.4 and Corollary 2.5.

Conversely, regard canonical instances as raw formulas. The raw calculus simulates each new inference using its equality transport, ordinary paired elimination, and the appropriate original case or equation-progress principle. The internally proved normalization equivalences convert canonical instances to raw substitution instances and back. Admissible cut converts retained contexts. The preceding term and formula equivalences give the nonliteral composite; uniqueness of normal forms

gives the literal canonical composite. This argument avoids assuming that a nonzero raw witness stays nonzero after normalization. □

A.2 Uniform evaluation is not an object-language operation

Each fixed finite scalar term denotes a primitive-recursive function. Uniform normalization on all term codes is nonetheless not primitive recursive. If it were, normalize the code of the eth unary scalar term on numeral input n and read the resulting numeral to obtain a primitive-recursive universal evaluator $V(e, n)$. Invalid codes can be assigned zero. The function $V(e, e) + 1$ would itself be primitive recursive and hence have a scalar-term index, giving the diagonal contradiction. The argument uses the scalar compilation theorem in the later appendices.

Thus normalization here is external computation on finite syntax. There is no internal quantification over arbitrary function descriptions and no new totality axiom in arithmetic. The optional fuel verifier makes the certificate predicate primitive recursive without asserting a uniform bound on the fuel needed for all true certificates.

Attribution and role of the following foundations

The next five appendices reproduce the pinned reference development from [6, 7, 8, 9], with its own explicit rule inventories, logical macros, internal arithmetic arguments and interpretation proofs. They are reference calculi and derived constructions, not additional primitive axioms of the three-instruction calculus. Their full inclusion makes the arithmetic bridge reviewable without relying on a claim of standard-model correctness. Names such as "old" and "new" within these attributed appendices refer to their inherited comparison, not the witness discharge innovation of this report.

B Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

B.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}\ \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F}\ \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x, y)}\ \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0,1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

B.2 Exact equivalence and the unused-witness construction

Lemma B.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem B.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi: \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t,Sz)$ under $\Gamma, E(t,0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t,y)$ and all retained data. Under $\Gamma, E(t,0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y} E(t,y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at Sz gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x,y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t,y)$ and binds only the unused coordinate u.

Proposition B.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \leq 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

B.3 Induction and numerical cases separate cleanly

Lemma B.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma B.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D are derivable without using the progress choice.

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

B.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \qquad \text{(Gate and padding)}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B), \\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B), \\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the fixed computations $p(1) = 1$ and

$p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z} M_z(A, B), \\ A \wedge_* B &:= \mathsf{D}_{u,z} G_z M_{p(z)}(A, B), \\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y} G_y A(x), \\ A \vee_* B &:= \exists_* x\, M_x(A, B), \\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z} G_z A(z), \\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

B.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$

$$\langle A; y.B \rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y}B \dashv\vdash \exists_* x(B(x,0) \Rightarrow_* \forall_* y\, B(x,y)). \qquad \text{(Paired composite)}$$

Forwards, DE opens its clause at a fresh a; under $B(a,0)$ that clause supplies $B(a,y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards, eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem B.2 preserves every native sequent.

B.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

C Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two

equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section B recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

C.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.

2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.

3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.

4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n,\vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem C.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

D Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [7] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

D.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{1}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n + 1) = 1, \qquad N(3n) = N(n). \tag{2}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

D.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{3}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n) + 1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n+2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{4}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{5}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (5) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{6}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (5); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

D.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m + n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{7}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (6), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{8}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{9}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \leq y$ is motivation, not a premise of the following argument.

D.4 Saturated iteration gives predecessor

We now need only the three equations (9). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (10) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (11) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \tag{12}$$

The base is (11). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \tag{13}$$

The first uses the comparator recurrence. The second uses (12) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \tag{14}$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \tag{15}$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (14) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

D.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

E From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [8]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix D, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

E.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{16}$$

$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{17}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x \, (x = 0 \vee x = S(Px)), \tag{18}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{19}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p} \, \left[(E(0, \vec{p}) \wedge \forall n (E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n \, E(n, \vec{p}) \right]. \tag{20}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (20) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0, \vec{p}), \qquad \Gamma, E(z, \vec{p}) \vdash E(Sz, \vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n, \vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

E.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u, 0, 1), \qquad K(u) = C(u, 1, Pu). \tag{21}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{22}$$

$$K(0) = 1, \qquad K(Sx) = x. \tag{23}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (16). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (22); symmetry and (19) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{24}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{25}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{26}$$

To prove the range assertion, split (18). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (17) also give, for all u, v,

$$C(u, v, v) = v. \tag{27}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{28}$$

If $u = 1 = S0$, (23) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

E.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{29}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{30}$$

Using these equations and (20), we now derive

$$0 + y = y, \tag{31}$$
$$(Sx) + y = S(x + y), \tag{32}$$
$$d(Sx, Sy) = d(x, y), \tag{33}$$
$$d(Sa, a) = 1. \tag{34}$$

Each induction below has a single equation as its induction formula.

For (31), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (32), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (32) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{35}$$

Indeed, both sides are $S(y + r)$.

For (33), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (34). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (33) and the induction hypothesis.

E.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C\big(\mathrm{sg}(d(x, y)),\ x,\ y + d(x, y)\big) = x. \tag{36}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (31) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (27).

For the step, assume (36) at y and abbreviate

$$r = d(x, y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (36) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (35) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (36). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{37}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (36). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (34) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a, b) = \mathrm{sg}\big(K(d(Sa, b))\big). \tag{38}$$

The sign, countdown, and residual reflection lemmas give

$$e(a, b) = 0 \quad \longleftrightarrow \quad a = b, \tag{39}$$

$$e(a, b) = 0 \ \vee \ e(a, b) = 1. \tag{40}$$

In detail, $e(a, b) = 0$ is equivalent to $K(d(Sa, b)) = 0$ by (25), which is equivalent to $d(Sa, b) = 1$ by (28), and then to $a = b$ by (37). The range assertion is (24) applied to $K(d(Sa, b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a, b) = 0$ branch, (39) gives $a = b$. In the $e(a, b) = 1$ branch, an assumption $a = b$ would give $e(a, b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a \, \forall b \, \big(a = b \vee (a = b \to \bot)\big) \tag{41}$$

is a theorem, not an extra hypothesis.

E.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s, t), \qquad b_\bot = 1, \tag{42}$$

$$b_{A \wedge D} = C(b_A, b_D, 1), \tag{43}$$

$$b_{A \vee D} = C(b_A, 0, b_D), \tag{44}$$

$$b_{A \to D} = C(b_A, b_D, 0). \tag{45}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$\big(b_A = 0 \vee b_A = 1\big) \quad \text{and} \quad \big(b_A = 0 \longleftrightarrow A\big). \tag{46}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (39)–(40). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (46) for every quantifier-free formula.

E.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (46), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation

induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

F Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [9]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

F.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a, b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v + 1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n + 1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n) + 1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a, b) = (a + b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \leq k^2 + a < (k+1)^2$ since $a \leq k$. Root uniqueness gives $q(\pi(a,b)) = a + b$, and therefore

$$L(\pi(a,b)) = a, \qquad H(\pi(a,b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

F.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u, \vec{p}) = \pi(SL(u), h^*(L(u), H(u), \vec{p})),$$

$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u, \vec{p})), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n, \vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

F.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section B give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

G Complete normalized acceptance checker

The following is the entire measured 2,273-byte module. The readable `code/checker.py` has exactly this Python AST after removal of its module docstring. The normalizer and all validation logic are included.

```
import json
import sys

def need(condition):
    if not condition:
        raise ValueError('invalid certificate')

def walk(a, f, d=0, form=None):
    if type(a) is int:
        need(not form)
        return f(a, d)
    need(type(a) is list)
    size = len(a)
    need(form is None or (size in ((1, 2) if form else (2, 3)) and (form or size == 3 or a[0] == 0)))
    a = [walk(b, f, d + (2 if size == 1 else int(size == 3 and i == 2)), size == 1) for i, b in enumerate(a)
    ]
    if size == 3:
        c, b, h = a
        if c == 0:
            return b
        if type(c) is list and len(c) == 2:
            return sub(h, walk([c[1], b, h], lambda i, d: i))
    return a

def syntax(a, n, form=False):

    def variable(i, d):
        need(-n <= i <= d)
        return i
    need(walk(a, variable, form=form) == a)

def shift(a, k):
    return walk(a, lambda i, d: i + k if i > d else i)

def sub(a, t, depth=0):
    return walk(a, lambda i, d: shift(t, d) if i == d + 1 else i - (i > d + 1), depth)

def prove(a, p, g, n):
    syntax(a, n, True)
    match p:
        case ['u', j, t, premise]:
            need(type(j) is int and 0 <= j < len(g))
            syntax(t, n)
            base, body = g[j]
            need(a == sub(body, t))
            if base:
                prove(base, premise, g, n)
        case ['e', body, fml, t, major, eq, branch]:
            prove([body], major, g, n)
            body = sub(body, -n - 1, 1)
            local = g + [(sub(body, 0), body)]
            prove([-n - 1, t], eq, local, n + 1)
            prove(fml, branch, local, n + 1)
            need(walk(fml, lambda i, d: t if i == -n - 1 else i) == a)
        case ['i', t, branch]:
            need(len(a) == 1)
            syntax(t, n)
            body = sub(a[0], t, 1)
            hyp = sub(body, -n - 1 if len(body) == 2 and t == 0 else 0)
            prove(sub(body, [0, -n - 1]), branch, g + [([], hyp)], n + 1)
```

```
        case _:
            raise ValueError('invalid proof')

def check(data):
    need(type(data) is dict and set(data) == {'goal', 'proof'})
    prove(data['goal'], data['proof'], [([], [1, 1])], 0)
if __name__ == '__main__':
    try:
        check(json.load(sys.stdin))
    except Exception:
        print('REJECT')
        sys.exit(1)
    print('ACCEPT')
```

H Reproduction and file map

The archive is self-contained. Python reproduction uses only the standard library; PDF building requires the standard TeX packages named in this single `report.tex` file. Run:

```
python3 verify_manifest.py
python3 reproduce.py
python3 code/checker.py < validation/examples/successor_congruence.json
python3 build_pdf.py
```

The report source embeds the mathematical appendices and code listing; it does not use external `input` files. The `reference/` directory preserves the attributed sources for comparison. The `validation/` directory contains untrusted proof producers, translations, an independent replay kernel and adversarial examples. The `evidence/` directory records exact measurements and executed results. The checksum manifest covers the delivered payload, excluding the manifest itself. Rebuilding the PDF may change its metadata and hash; the manifest describes the original delivered archive.

References

[1] V. Reshetnikov, ed., *Concise presentations of intuitionistic primitive recursive arithmetic*, ProveIt, frozen revision `0ebd0c6b358f38bb0ad2883faf775421599c8aae`. Repository snapshot.

[2] *iPRA variable substitution*, incoming archive `ipra_variable_substitution_2026-11-11.zip`, especially `paper/variable_substitution_proof.tex`, in [1]. Source of the two-row naming packet; its equality rule and witness elimination remain separate.

[3] *iPRA with uniform clause use*, incoming archive `ipra_uniform_clauses.zip`, in [1]. Source of zero/singleton-premise clauses and the initial reflexivity clause.

[4] *Five-rule iPRA with a single evaluation schema*, incoming archive `ipra_clausal_single_evaluation_2026-11-11.zip`, in [1]. The complete source is preserved as `reference/single_evaluation.py`.

[5] *Full normalization*, `Research/full_normalization`, in [1], especially `sections/normalization.tex` and the compound-envelope repair.

[6] *Certified zero*, `Research/certified_zero/reference`, in [1]. Sources: `inherited_choice_basis.tex` and `inherited_ordinary_bridge.tex`. The attributed

appendices reproduce these foundations to make the complete logical interface and both interpretation composites available in this report.

[7] *Scalar predecessor bootstrap*, `Research/certified_zero/reference/scalar_predecessor.tex`, in [1]. This is the inherited one-hole scalar construction, including the factor map, comparator and saturated iteration proof.

[8] *Equality reflection*, `Research/certified_zero/reference/equality_reflection.tex`, in [1]. Internal reflection of actual equality and compilation of quantifier-free formulas, not an assumed arithmetic equality test.

[9] *Pairing and primitive recursion*, `Research/certified_zero/reference/primitive_recursion.tex`, in [1]. The inherited progress-closure development of square pairing and the scalar primitive-recursion compiler.